Implementation of Single-Cycle RISC-V Processor in Verilog

Hassan Raza (2024229) , lol was just testing the limits of the system

Computer Organization and Assembly Language Lab
Date: 8 May 2026

***Abstract*—RISC-V is a modern open-source Instruction Set Architecture (ISA) designed to provide simplicity, flexibility, modularity, and scalability in processor design. In this project, a Single-Cycle RISC-V Processor was implemented using Verilog HDL. The processor executes one instruction completely within a single clock cycle. The implemented processor supports arithmetic, logical, memory-access, and branch instructions including ADD, SUB, AND, OR, ADDI, LW, SW, and BEQ. Major hardware components including Program Counter, Instruction Memory, Register File, Control Unit, ALU Control Unit, Arithmetic Logic Unit, Data Memory, Immediate Generator, Multiplexers, and Adders were integrated into a complete datapath. Simulation and testing were performed using a Verilog testbench to verify correct processor functionality.**

***Index Terms*—RISC-V, Verilog HDL, Single-Cycle Processor, ALU, Datapath, Computer Architecture**

I. Introduction

RISC-V is an open-source Instruction Set Architecture (ISA) developed at the University of California, Berkeley. It provides a simple, modular, and extensible processor architecture for modern hardware design.

RISC-V follows the Reduced Instruction Set Computer (RISC) philosophy where instructions are simple and optimized for efficient hardware implementation. Because of its open-source nature, RISC-V is increasingly used in research, embedded systems, IoT devices, and processor development.

In this project, a Single-Cycle RISC-V Processor was implemented using Verilog HDL. In a single-cycle architecture, each instruction completes execution within one clock cycle. Although this design is not optimized for maximum performance, it provides a clear understanding of datapath organization and processor operation.

The processor supports arithmetic, logical, memory, and branch instructions and integrates all major datapath components into a complete processor architecture.

II. Objectives

The objectives of this project are:

- To understand RISC-V architecture and instruction formats.
- To understand instruction fetch, decode, execute, memory, and write-back stages.
- To implement processor modules using Verilog HDL.
- To design a complete single-cycle processor datapath.
- To simulate and verify processor functionality using a testbench.

III. Background Theory

A. RISC-V Architecture

RISC-V is based on the Reduced Instruction Set Computer architecture. The ISA uses simple and fixed instruction formats that simplify hardware implementation.

Key features of RISC-V include:

- Simple and modular architecture
- Open-source design
- Extensible instruction set
- Efficient hardware implementation
- High portability

B. Single-Cycle Processor

In a single-cycle processor, every instruction executes completely within one clock cycle.

Major stages include:

1) Instruction Fetch
2) Instruction Decode
3) Execute
4) Memory Access
5) Write Back

Advantages include simple control logic and easy debugging, while disadvantages include longer clock cycles.

IV. Instruction Formats

A. R-Type Instructions

R-type instructions are used for arithmetic and logical operations.

Supported instructions:

- ADD
- SUB
- AND
- OR

B. I-Type Instructions

I-type instructions use immediate operands.

Supported instructions:

- ADDI
- LW

C. S-Type Instructions

S-type instructions are used for memory store operations.

Supported instruction:

- SW

D. SB-Type Instructions

SB-type instructions are used for branch operations.
Supported instruction:

- BEQ

V. Implemented Instructions

TABLE I
IMPLEMENTED INSTRUCTIONS

Instruction	Type	Description
ADD	R-type	Addition
SUB	R-type	Subtraction
AND	R-type	Logical AND
OR	R-type	Logical OR
ADDI	I-type	Add Immediate
LW	I-type	Load Word
SW	S-type	Store Word
BEQ	SB-type	Branch if Equal

VI. Processor Architecture

The processor follows the standard single-cycle datapath architecture.

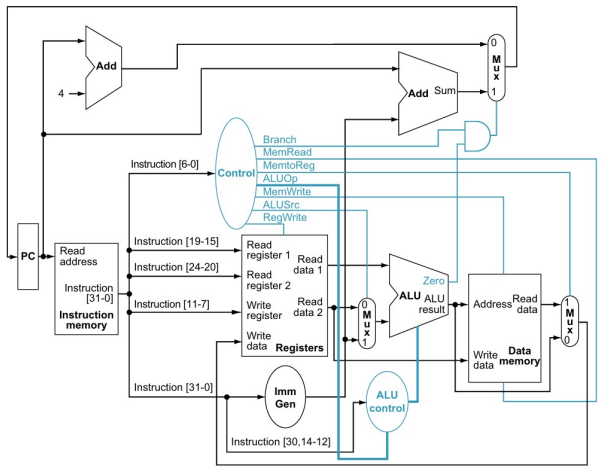

Fig. 1. Single-cycle processor datapath

The datapath integrates:

- Program Counter
- Instruction Memory
- Decoder
- Register File
- ALU
- Data Memory
- Multiplexers
- Adders
- Control Unit
- ALU Control Unit

VII. Modules Description

A. Program Counter

The Program Counter stores the address of the next instruction.

Functions include:

- Holding current instruction address
- Updating instruction sequence
- Supporting branch operations

B. Instruction Memory

Instruction memory stores machine instructions executed by the processor.

Sample instructions include:

- addi x1, x0, 5
- addi x2, x0, 10
- add x3, x1, x2
- sub x4, x3, x1
- and x5, x3, x2
- or x6, x4, x2
- sw x3, 0(x0)
- lw x7, 0(x0)
- beq x7, x3, 4

C. Decoder

The decoder extracts instruction fields from the 32-bit instruction.

TABLE II
INSTRUCTION FIELDS

Field	Bit Range
Opcode	[6:0]
rd	[11:7]
funct3	[14:12]
rs1	[19:15]
rs2	[24:20]
funct7	[31:25]

D. Control Unit

The control unit generates control signals according to the instruction opcode.

Generated signals include:

- ALUSrc
- MemtoReg
- RegWrite
- MemRead
- MemWrite
- Branch
- ALUOp

TABLE III
OPCODE HANDLING

Opcode	Instruction Type
0110011	R-Type
0000011	LW
0100011	SW
1100011	BEQ
0010011	ADDI

E. Immediate Generator

The Immediate Generator extracts and sign-extends immediate values.

Supported formats include:

- I-Type
- S-Type
- B-Type

For I-type instructions:

$$Immediate = \{20\{instruction[31]\}, instruction[31:20]\}$$

F. Register File

The Register File stores operands and intermediate data.

Features include:

- 32 general-purpose registers
- Two read ports
- One write port
- Register x0 always equals zero

G. ALU Control Unit

The ALU Control Unit determines the required ALU operation.

TABLE IV
ALU OPERATIONS

ALU Control	Operation
0000	AND
0001	OR
0010	ADD
0110	SUB

H. Arithmetic Logic Unit

The Arithmetic Logic Unit performs arithmetic and logical operations.

Supported operations include:

- Addition
- Subtraction
- AND
- OR

Zero flag equation:

$$Zero = (Result == 0)$$

I. Data Memory

The Data Memory performs load and store operations.

Features include:

- 32 memory locations
- Synchronous write
- Asynchronous read

J. Multiplexers

Multiplexers select data sources in the datapath.

Implemented multiplexers include:

- ALU Source Multiplexer
- Memory-to-Register Multiplexer

VIII. Top Module Integration

The `single_cycle_processor` module integrates all datapath components into a complete processor architecture.

Execution flow:

1) PC provides instruction address
2) Instruction memory fetches instruction
3) Decoder extracts instruction fields
4) Control unit generates signals
5) Register file provides operands
6) ALU performs operations
7) Data memory handles memory instructions
8) Results are written back to registers

IX. Branch Operation

The processor supports the BEQ branch instruction.

Branch operation:

- ALU subtracts two register values
- Zero flag determines equality
- Branch target address is calculated
- PC updates accordingly

$$PC_{next} = \begin{cases} BranchTarget, & \text{if Branch AND Zero = 1} \\ PC + 4, & \text{otherwise} \end{cases}$$

X. Testbench

A Verilog testbench was developed to verify processor functionality.

The testbench performs:

- Clock generation
- Reset generation
- Output monitoring
- Waveform dumping

Monitored outputs include:

- Program Counter
- Current Instruction
- ALU Result
- Register Outputs
- Data Memory Output

XI. Simulation Results

Simulation verified successful execution of all implemented instructions.

TABLE V
SIMULATION RESULTS

Instruction	Result
addi x1, x0, 5	x1 = 5
addi x2, x0, 10	x2 = 10
add x3, x1, x2	x3 = 15
sub x4, x3, x1	x4 = 10
and x5, x3, x2	x5 = 10
or x6, x4, x2	x6 = 10
lw x7, 0(x0)	x7 = 15

A. Waveform Output

The simulation waveform displays the temporal behavior of key processor signals throughout the execution cycle. The waveform confirms proper signal propagation through the datapath and correct instruction sequencing.

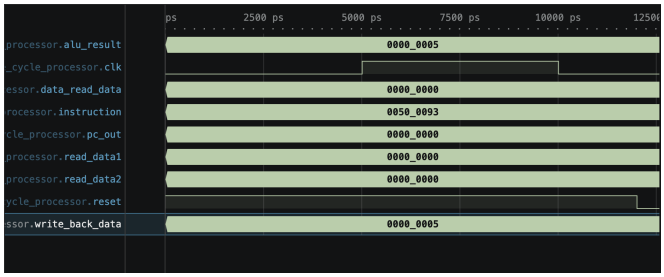

Fig. 2. Processor simulation waveform output showing ALU results, clock, instruction data, register outputs, and data memory operations

XII. Performance Analysis

The implemented processor successfully executes all supported instructions in a single clock cycle. Since all stages of execution occur simultaneously within one cycle, the processor design remains simple and easy to debug.

The processor correctly handled:

- Arithmetic computations using ADD and SUB instructions
- Logical operations using AND and OR instructions
- Immediate arithmetic using ADDI
- Memory access operations using LW and SW
- Branch decision making using BEQ

The branch mechanism was verified by comparing register values and updating the Program Counter accordingly.

Waveform analysis confirmed correct instruction sequencing and datapath operation.

XIII. Design Challenges and Debugging

During implementation, several challenges were encountered while integrating processor modules into a complete datapath.

One major challenge was synchronization between the control unit and datapath modules. Incorrect control signals initially caused improper ALU operations and write-back behavior.

Another challenge involved immediate generation for branch instructions. Proper sign extension and branch target calculation were necessary for correct branching behavior.

Memory addressing also required debugging because the memory array was word-addressable.

Branch implementation required verification of:

- ALU subtraction operation
- Zero flag generation
- Branch target calculation
- Program Counter update logic

Simulation waveforms and monitored outputs were used extensively during debugging.

XIV. Advantages

Advantages of the implemented processor include:

- Simple datapath organization
- Easy debugging and verification
- Straightforward control logic
- One-cycle instruction execution
- Useful for educational purposes

XV. Limitations

Limitations include:

- Long clock cycle duration
- Lower performance compared to pipelined processors
- Inefficient hardware utilization

XVI. Datapath Execution Flow

The execution of an instruction inside the single-cycle processor follows a fixed datapath sequence. Every instruction passes through all stages of execution during one clock cycle.

A. Instruction Fetch

During the fetch stage, the Program Counter (PC) provides the address of the instruction to the instruction memory. The instruction memory outputs the corresponding 32-bit instruction.

The PC value is also incremented by 4 using an adder to point toward the next instruction.

B. Instruction Decode

In the decode stage, the decoder extracts the instruction fields including opcode, source registers, destination register, funct3, and funct7.

The control unit analyzes the opcode and generates control signals required for datapath operation.

At the same time, the register file reads operand values from the source registers.

C. Execution Stage

The ALU performs arithmetic or logical operations based on the ALU control signal.

For immediate instructions, the ALU receives the immediate value through the ALUSrc multiplexer.

For branch instructions, the ALU compares register values and generates the zero flag.

D. Memory Access Stage

For load and store instructions, the ALU output acts as the memory address.

- LW reads data from memory
- SW writes data into memory

The memory module performs operations according to MemRead and MemWrite control signals.

E. Write Back Stage

The final stage writes data back into the register file.

The MemtoReg multiplexer selects whether the write-back value comes from:

- ALU result
- Data memory output

This completes execution of the instruction within one clock cycle.

XVII. Control Signal Analysis

Control signals are essential for coordinating datapath operations inside the processor.

The control unit generates signals depending on the instruction opcode.

TABLE VI
MAIN CONTROL SIGNALS

Signal	Function
ALUSrc	Selects ALU operand source
MemtoReg	Selects write-back source
RegWrite	Enables register writing
MemRead	Enables memory reading
MemWrite	Enables memory writing
Branch	Enables branch operation
ALUOp	Determines ALU operation type

For R-type instructions:

- Register operands are selected
- ALU performs arithmetic/logical operations
- Result is written back to the register file

For load instructions:

- Immediate value is added to base address
- Data memory performs read operation
- Loaded value is written back into a register

For store instructions:

- ALU calculates memory address
- Register data is written into memory

For branch instructions:

- ALU compares register values
- Zero flag determines branch decision
- Program Counter updates accordingly

XVIII. Conclusion

A complete Single-Cycle RISC-V Processor was successfully implemented using Verilog HDL.

Major datapath components including Program Counter, Instruction Memory, Register File, ALU, Control Unit, Immediate Generator, and Data Memory were integrated into a complete processor architecture.

Simulation results verified successful execution of arithmetic, logical, memory, and branch instructions.

The project provided strong understanding of:

- Processor datapath organization
- Instruction execution flow
- Control signal generation
- Hardware implementation using Verilog HDL
- RISC-V instruction formats

XIX. Future Improvements

Future enhancements may include:

- Pipelined processor architecture
- Hazard detection unit
- Forwarding unit
- Cache memory implementation
- Support for additional RISC-V instructions
- Performance optimization

References

[1] RISC-V Foundation, "The RISC-V Instruction Set Manual."

[2] David Patterson and John Hennessy, *Computer Organization and Design*, Morgan Kaufmann Publishers.

[3] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*.

[4] Lab Manual: Implementation of RISC-V Processor in Verilog.